

棚卸初期化仕様書

1. 概要

本書は、[設定](#) → [棚卸初期化](#) 画面で行う以下2処理の仕様を定義する。

1. 棚卸Excel/CSV取込（在庫反映）
2. 在庫・計画在庫・進度の初期化計算

本仕様は **現行実装** ([pm_backend/apps/production/views.py](#) + [pm_backend/apps/production/inventory/stocktake_initializer.py](#)) を正とする。

2. 目的

- 棚卸実績を基準に在庫関連値を初期化する
- 初回投入や棚卸後リセット時に、在庫・計画在庫・進度を一貫再構築する

3. 画面仕様（StocktakeInit.vue）

対象画面:

- [pm-ui/src/views/settings/StocktakeInit.vue](#)

画面フロー:

1. テンプレートDL
2. 棚卸ファイル取込（在庫反映）
3. 初期化計算（在庫・計画在庫・進度）

権限:

- UI権限 [settings/edit](#) が必要（バックエンドでの権限判定は行わない）

3.1 棚卸日入力値の扱い（重要）

- Step1（取込）も Step2（初期化計算）も、**実行時点の画面入力** [stocktakeDate](#) を送信する。
- Step1の結果（[baseline_date](#)）を画面内部で固定保持して Step2 に自動引き継ぐ仕様ではない。
- そのため、**Step1後に棚卸日を変更した場合、Step2は変更後の棚卸日で実行される。**

運用ルール:

- Step1完了後は棚卸日を変更せずにStep2を実行すること。
- 変更した場合は、同じ棚卸日でStep1をやり直してからStep2を実行すること。

4. 業務ルール

- 基準日（[baseline_date](#)）は [stocktake_date](#) を採用する。
- **絶対基準日ルール:** [stocktake_date](#) より前（[plan_date](#) < [stocktake_date](#)）のデータは初期化計算に使用しない。
- [line_backlog.stock_qty](#) は整数管理のため、棚卸数量の小数は切り捨てで反映する。

- 同一品番が複数行ある場合はサーバー側で合算する。
- ルーティング未設定品番は `line_backlog` へ反映できず、レスポンスの `unmapped_product_codes` で通知する。
- `upper_bound_date`（前々営業日）は返却されるが、棚卸初期化の基準日決定には使わない。

5. API仕様

5.1 テンプレートDL

- `GET /api/line-backlogs/download_stocktake_template/`
- 返却: `xlsx`

テンプレート列:

- 製品番号
- 製品名
- 数量

5.2 棚卸取込

- `POST /api/line-backlogs/import_stocktake_excel/`
- Content-Type: `multipart/form-data`

リクエスト:

- `file`（必須）: `.xlsx` or `.csv`
- `stocktake_date`（必須）: `YYYY-MM-DD`
- `location`（任意）: 既定 `MAIN`

入力列（xlsx/csv共通）:

- 製品番号（必須）
- 数量（必須）
- 製品名 は任意（取込処理では使用しない）

処理:

1. ファイル読込、製品番号 単位で数量を合算
2. 未登録品番検証（存在時はエラー）
3. `t_stock_allocation` を作成/更新
 - 既存行は `current_stock`, `reserved_qty` のみ更新
4. `line_backlog` (`plan_date=stocktake_date`, `sequence_no=0`) を作成/更新
 - `is_stocktake_fix=True`
 - `stock_qty` のみ棚卸値を反映
 - `planned_stock_qty` は **0を保持**（Step2の初期化計算で設定）
 - `progress_qty`, `planned_progress_qty` は 0 にリセット

レスポンス主要項目:

- `stocktake_date`
- `baseline_date` (=stocktake_date)

- `upper_bound_date`
- `allocation_created, allocation_updated`
- `backlog_created, backlog_updated`
- `unmapped_product_codes`

5.3 初期化計算

- `POST /api/line-backlogs/initialize_stocktake/`

リクエスト:

- `stocktake_date` (必須) : YYYY-MM-DD
- `end_date` (必須) : YYYY-MM-DD

処理:

1. `baseline_dt = stocktake_date`
2. `ensure_stocktake_backlogs(baseline_dt)` で基準日行を補完
3. 対象ライン抽出 (`plan_date` が `baseline_dt..end_date`)
 - `plan_date < baseline_dt` のデータは対象外 (不使用)
4. `initialize_planned_stock(line_ids, baseline_dt)` を実行
5. 各ラインで `recalculate_inventory_from_stocktake(line_id, baseline_dt, end_date)` を実行
6. `initialize_progress_from_stocktake(baseline_dt)` で基準日進度を初期化
7. 各ラインで `recalculate_progress_from_stocktake(line_id, baseline_dt, end_date)` を実行

レスポンス主要項目:

- `stocktake_date, baseline_date, upper_bound_date, end_date`
- `line_count`
- `baseline_backlog_created, baseline_backlog_updated`
- `baseline_unmapped_product_codes`
- `updated_count, final_count, child_count`

エラー条件:

- 日付フォーマット不正
- `end_date < baseline_date`
- 計算対象期間に `line_backlog` が存在しない

6. 初期化計算ロジック

6.1 在庫 (stock_qty)

- `is_stocktake_fix=True` 行をアンカーとして採用
- `plan_date < baseline_date` の在庫値は参照しない
- 基準日以降を日次カスケードで再計算
- 親実績/仕損・仕損調整を反映

6.2 計画在庫 (planned_stock_qty)

- 基準日で `initialize_planned_stock` により初期値を設定
 - 実在庫からパイプライン需要を差し引く
 - パイプライン需要のLTは `RoutingStep.lead_time_days` → `Line.lead_time_days` → `BOMItem.lead_time_days` の優先順で解決
 - 最終品/ライン最終品は、基準日+1..+LT の自需要（finalはfirm優先、line-finalはorder）をパイプライン需要として差し引く
- `plan_date < baseline_date` の計画在庫値は参照しない
- その後、基準日以降を日次カスケードで再計算

6.3 進度（`progress_qty / planned_progress_qty`）

- BOMをトポロジカルソート（上位→下位）
- 基準日（アンカー）：
 - 最終品: `progress = stock_qty - 当日需要(LineDemand)`
 - 子部品: `progress = stock_qty + Σ(親進度(需要控除後) × BOM数量) - 当日需要(LineDemand)`
- 親進度は全ライン合算で扱う
- 基準日翌日以降は日次連鎖で再計算
 - `progress_qty = 前日進度 + 実績 - 需要(LineDemand) + 調整 + 仕損調整`
 - `planned_progress_qty = 前日計画進度 + 計画 - 需要(LineDemand) + 調整 + 仕損調整`
- 基準日より前の進度値は初期化計算に使用しない

7. 実装ファイル

バックエンド:

- `pm_backend/apps/production/views.py`
- `pm_backend/apps/production/inventory/stocktake_initializer.py`
- `pm_backend/apps/production/models_line_backlog.py` (`is_stocktake_fix`)

フロントエンド:

- `pm-ui/src/views/settings/StocktakeInit.vue`
- `pm-ui/src/api/resources/lineBacklogs.js`

8. 運用注意

- 初期化計算前に、生産計画を必要期間（最大LTを考慮）まで投入すること。
- `unmapped_product_codes` が返った場合、該当品番は `line_backlog` 初期化対象外となるため、ルーティング整備後に再実行すること。
- Step1→Step2の間で棚卸日を変更しないこと（変更時はStep1再実行）。